ASSESSMENT TOOL 2

ITA0609 - Machine Learning

Sameer Ahmed G

192421154

1. Pseudocode Algorithm for Data Cleaning and Transformation of Time-Series Data

Aim

To design an efficient pseudocode algorithm for cleaning and transforming time-series data by handling missing timestamps, interpolating missing values, removing outliers, resampling the data at fixed intervals, indexing observations by time, filtering invalid entries, and maintaining chronological consistency.

Introduction

Time-series data consists of observations collected at different points in time. Examples include temperature readings from IoT sensors, stock prices, electricity consumption, patient heart rate measurements, and weather observations. Real-world time-series datasets frequently contain missing timestamps, missing values, duplicate records, incorrect measurements, outliers, and irregular sampling intervals. Before applying machine learning or statistical models, the data must therefore be cleaned and transformed into a consistent chronological structure. The proposed algorithm uses time-based indexing, validation, interpolation, outlier detection, sorting, and resampling to produce a reliable and uniformly sampled time-series dataset.

Input

A raw time-series dataset containing at least:

- Timestamp
- One or more numerical measurements
- Optional categorical or identification attributes

Output

A cleaned and transformed time-series dataset with:

- Valid timestamps
- Chronological ordering
- No duplicate timestamps
- Missing values handled
- Outliers removed or corrected

- Fixed sampling frequency
- Time-based indexing

Pseudocode

```
ALGORITHM CleanAndTransformTimeSeries

INPUT:
    Raw time-series dataset D
    Timestamp column T
    Numerical value columns V
    Required resampling interval R
    Outlier threshold Z

OUTPUT:
    Cleaned time-series dataset C

BEGIN
    STEP 1: LOAD DATA
        Read dataset D
        IF D is empty THEN
            Display "No data available"
            STOP
        END IF
    STEP 2: VALIDATE COLUMNS
        Check whether timestamp column T exists
        IF timestamp column does not exist THEN
            Display "Invalid dataset"
            STOP
        END IF
```

Check whether required numerical columns V exist

IF required columns are missing THEN

Display "Required columns missing"

STOP

END IF

STEP 3: CONVERT TIMESTAMP

Convert T into standard datetime format

For each record:

IF timestamp cannot be converted THEN

Mark timestamp as invalid

END IF

Remove records having invalid timestamps

STEP 4: REMOVE INVALID VALUES

For every numerical feature X in V:

Convert X into numeric format

Identify invalid or non-numeric values

Replace invalid values with missing values

Remove records containing invalid timestamps

STEP 5: REMOVE DUPLICATE RECORDS

Identify duplicate rows

Remove exact duplicate records

Identify duplicate timestamps

IF multiple records have the same timestamp THEN

Aggregate values using mean/median

OR retain the most reliable observation

END IF

STEP 6: INDEX DATA BY TIME

Set timestamp column T as the DataFrame index

STEP 7: SORT CHRONOLOGICALLY

Sort all records according to timestamp

Ensure:

timestamp[i] <= timestamp[i+1]

STEP 8: VALIDATE TIME RANGE

Determine minimum timestamp

Determine maximum timestamp

Remove observations outside the valid

analysis period if required

STEP 9: DETECT MISSING TIMESTAMPS

Generate expected timestamp sequence

between minimum and maximum timestamp

Compare expected timestamps with

timestamps present in D

Identify missing timestamps

STEP 10: CREATE COMPLETE TIME INDEX

Reindex dataset using the complete

expected timestamp sequence

Missing timestamps now contain

missing values

STEP 11: INTERPOLATE MISSING VALUES

FOR each numerical feature X:

IF missing values occur between

valid observations THEN

Apply time-based interpolation

END IF

END FOR

STEP 12: HANDLE REMAINING MISSING VALUES

Check for missing values remaining

at the beginning or end of the series

IF appropriate THEN

Apply forward filling or backward filling

ELSE

Remove records that cannot be reliably

reconstructed

END IF

STEP 13: DETECT OUTLIERS

FOR each numerical feature X:

Calculate mean and standard deviation

Calculate Z-score for each observation

IF absolute(Z-score) > Z THEN

Mark observation as outlier

END IF

END FOR

STEP 14: HANDLE OUTLIERS

FOR each detected outlier:

IF outlier is caused by measurement error THEN

Replace using interpolation

OR replace using rolling median

ELSE

Retain the observation if it represents

a genuine extreme event

END IF

END FOR

STEP 15: RESAMPLE DATA

Select required fixed interval R

Resample time-series data using R

For example:

1-minute → 5-minute

5-minute → 1-hour

hourly → daily

Apply appropriate aggregation:

Mean for continuous measurements

Sum for quantities

Maximum for peak values

Minimum for minimum values

STEP 16: FINAL VALIDATION

Check for:

Missing timestamps

Duplicate timestamps

Remaining missing values

Invalid numerical values

Incorrect ordering

```
        Unexpected frequency
    IF any serious error remains THEN
        Display validation error
        Return to appropriate cleaning step
    END IF

  STEP 17: ENSURE CHRONOLOGICAL CONSISTENCY
    Sort final dataset by timestamp
    Verify that all timestamps
    are in ascending order

  STEP 18: RETURN CLEAN DATA
    Store cleaned dataset as C
    Return C
END
```

Explanation of Major Steps

1. Timestamp Conversion

The timestamp is the most important attribute in time-series data. Different files may use formats such as 2026-08-23 10:30:00, 23/08/2026 10:30, or Unix timestamps. All timestamps must be converted into one standard datetime representation. Invalid timestamps are removed because they cannot be correctly positioned in the time sequence.

2. Time-Based Indexing

After validating timestamps, the timestamp column is used as the index. Time-based indexing allows efficient operations such as selecting a particular date range, calculating rolling statistics, detecting gaps, and resampling the data.

3. Chronological Sorting

Time-series observations must be arranged in chronological order. Sorting the dataset by timestamp ensures that interpolation, rolling calculations, and resampling operate correctly.

4. Missing Timestamp Detection

Missing timestamps are different from missing values. For example, if sensor readings are expected every five minutes, the sequence should be:

10:00

10:05

10:10

10:15

10:20

If 10:10 is absent, the timestamp itself is missing. The algorithm generates the expected sequence and compares it with the actual timestamps to identify such gaps.

5. Missing Value Interpolation

After creating the complete time index, missing measurements can be estimated using interpolation. Time-based interpolation is appropriate because the values before and after a missing observation provide information about the expected value.

For example:

10:00 → 20

10:05 → Missing

10:10 → 30

Linear interpolation estimates:

$$
Value_{10:05}=\frac{20+30}{2}=25
$$

For more complex data, spline interpolation or rolling-window methods can be considered.

6. Outlier Detection

Outliers are observations that differ significantly from the normal pattern. One method for detecting outliers is the Z-score:

$$
Z=\frac{x-\mu}{\sigma}
$$

If

$$
|Z|>3
$$

the observation can be considered a potential outlier under the common three-standard-deviation rule. However, domain knowledge should be used before removing it because an extreme value may represent a genuine event rather than an error.

7. Outlier Handling

Outliers caused by sensor errors can be replaced using interpolation or a rolling median. Genuine extreme events should generally be retained because removing them could destroy important information. For example, an unusually high electricity reading may be a sensor error, but it could also represent a genuine peak-demand event.

8. Resampling

Time-series data may be recorded at irregular or unnecessarily high frequencies. Resampling converts the observations to a fixed interval.

For example:

Raw Data: Every 1 minute

Required Data: Every 1 hour

For continuous measurements such as temperature, the hourly mean may be calculated. For electricity consumption, values may be summed when each observation represents energy consumed during the interval.

Efficiency Considerations

Efficiency is important when processing millions of time-series records. Vectorized Pandas operations should be preferred over Python loops wherever possible. The timestamp column should be converted once and used as the index. Duplicate records should be removed before expensive transformations. Resampling and interpolation should be performed using optimized time-series functions. For extremely large datasets, data can be processed in chunks before being combined. Only required columns should be loaded into memory.

Final Validation

After processing, the algorithm must verify that the dataset satisfies all requirements. The timestamps should be unique, correctly sorted, and equally spaced according to the required frequency. Missing values should be checked, invalid numerical values should be removed or corrected, and the final number of observations should be compared against the expected time range.

Conclusion

The proposed pseudocode provides a complete and efficient procedure for cleaning and transforming time-series data. It handles invalid timestamps, duplicate observations, missing timestamps, missing values, outliers, irregular sampling, and chronological inconsistencies. Time-based indexing and sorting ensure correct temporal ordering, while interpolation reconstructs missing measurements and resampling produces a fixed sampling interval. Final validation ensures that the resulting dataset is suitable for machine learning, statistical analysis, forecasting, and visualization.

2. Pseudocode Algorithm for Feature Selection

Aim

To design a detailed pseudocode algorithm for selecting significant features from a dataset by calculating feature correlations, removing redundant and irrelevant attributes, normalizing the data, applying threshold-based filtering, and validating the selected feature subset to improve model performance and reduce dimensionality.

Introduction

Feature selection is the process of identifying the most useful input attributes for a machine learning model while removing irrelevant, redundant, or noisy features. A dataset containing a large number of attributes can increase computational cost, introduce unnecessary noise, cause overfitting, and reduce model interpretability. Feature selection addresses these problems by retaining only the features that provide meaningful information for predicting the target variable. Correlation analysis is one commonly used technique for identifying relationships between numerical features. Features that are highly correlated with each other may contain redundant information, while features with very weak relationships with the target may be removed. Normalization and threshold-based filtering further improve the selection process.

Input

A dataset (D) containing:

- Feature matrix (X)
- Target variable (Y)
- Correlation threshold (T_c)
- Target relevance threshold (T_r)

Output

A reduced feature dataset containing significant and non-redundant attributes.

Pseudocode

ALGORITHM FeatureSelection

INPUT:

Dataset D

Target variable Y

Feature set X

Correlation threshold Tc

Relevance threshold Tr

OUTPUT:

Selected feature set X_selected

BEGIN

STEP 1: LOAD DATASET

Read dataset D

IF dataset is empty THEN

Display "Dataset unavailable"

STOP

END IF

STEP 2: IDENTIFY FEATURES AND TARGET

Separate target variable Y

Separate input features X

Identify numerical features

Identify categorical features

STEP 3: REMOVE DUPLICATE RECORDS

Find duplicate rows

Remove duplicate records

STEP 4: HANDLE MISSING VALUES

FOR each feature X_i:

Calculate percentage of missing values

IF missing percentage is very high THEN

Remove feature X_i

ELSE

Impute missing values

END IF

END FOR

STEP 5: HANDLE INVALID VALUES

Check numerical ranges

Detect invalid or impossible values

Correct or remove invalid observations

STEP 6: ENCODE CATEGORICAL FEATURES

Convert categorical variables

into numerical representation

Use appropriate encoding method

STEP 7: NORMALIZE FEATURES

FOR each numerical feature X_i:

Calculate minimum and maximum

Apply Min-Max normalization:

X'_i =

(X_i - X_min) /

(X_max - X_min)

END FOR

STEP 8: CALCULATE FEATURE-TARGET CORRELATION

FOR each feature X_i:

Calculate correlation between X_i and Y

Store absolute correlation value

END FOR

STEP 9: REMOVE IRRELEVANT FEATURES

FOR each feature X_i:

IF absolute correlation(X_i,Y) < Tr THEN

Mark X_i as irrelevant

END IF

END FOR

Remove all irrelevant features

STEP 10: CALCULATE FEATURE-FEATURE CORRELATION

Calculate correlation matrix

for remaining features

STEP 11: IDENTIFY REDUNDANT FEATURES

FOR every pair (X_i, X_j):

IF absolute correlation(X_i,X_j) >= Tc THEN

Mark one feature as redundant

Retain the feature having

stronger correlation with target

END IF

END FOR

STEP 12: CREATE SELECTED FEATURE SET

Add all relevant and

non-redundant features

Store as X_selected

STEP 13: VALIDATE FEATURE SET

Train baseline model using
original feature set
Train model using X_selected
Calculate:
- Accuracy
- Precision
- Recall
- F1-score

STEP 14: COMPARE MODEL PERFORMANCE

IF performance using X_selected
is equal to or better than baseline THEN
- Accept selected feature set

ELSE
- Review thresholds
- Re-evaluate removed features
- Repeat feature selection

END IF

STEP 15: CHECK DIMENSIONALITY REDUCTION

Calculate:
Original feature count
Selected feature count
Calculate reduction percentage

STEP 16: FINAL VALIDATION

Check for:

Missing values

Duplicate features

Highly correlated features

Irrelevant features

IF validation is successful THEN

Return X_selected

ELSE

Display validation error

END IF

END

Step 1: Data Preparation

The first stage is to load the dataset and separate the input features from the target variable. Duplicate records should be removed because repeated observations can bias correlation calculations and model evaluation.

Step 2: Handling Missing Values

Missing values must be handled before calculating correlations. Numerical attributes can be imputed using the mean or median, while categorical variables can be replaced using the mode or an appropriate category. Features containing an extremely high percentage of missing values may be removed because reliable information cannot be obtained from them.

Step 3: Handling Invalid Data

Invalid values should be identified using domain-specific rules. For example, an age value of -10 is invalid, and a percentage greater than 100 may also be invalid. Such values should be corrected, treated as missing, or removed depending on the context.

Step 4: Encoding Categorical Features

Correlation calculations generally require numerical representations. Categorical variables can therefore be transformed using methods such as one-hot encoding or ordinal encoding, depending on whether the categories have a meaningful order.

Step 5: Normalization

Normalization ensures that numerical features are placed on comparable scales. Min-Max normalization is given by:

$$
X'=\frac{X-X_{min}}{X_{max}-X_{min}}
$$

This transforms values approximately into the range (0) to (1). Standardization can alternatively be used:

$$
Z=\frac{X-\mu}{\sigma}
$$

Normalization is particularly important for algorithms that depend on numerical distances or gradient-based optimization.

Step 6: Feature-Target Correlation

The correlation between each feature and the target is calculated. Pearson correlation can be used for continuous numerical variables.

$$
r_{XY}=
\frac{\operatorname{Cov}(X,Y)}
{\sigma_X\sigma_Y}
$$

The correlation value ranges from (-1) to (+1). A value close to (+1) indicates a strong positive relationship, while a value close to (-1) indicates a strong negative relationship. A value close to zero indicates a weak linear relationship.

Step 7: Threshold-Based Filtering

A relevance threshold is selected to remove features that have very weak relationships with the target. For example, suppose:

$$
T_r=0.10
$$

Any feature with:

$$
|r|<0.10
$$

can be considered a candidate for removal.

The threshold should not be chosen blindly because a feature with low linear correlation may still contain nonlinear predictive information. Therefore, threshold filtering should be followed by model validation.

Step 8: Feature-Feature Correlation

After removing irrelevant features, correlations between the remaining features are calculated. This identifies redundant features that provide almost the same information.

For example:

Feature Pair	**Correlation**
Age – Income	0.25
Income – Spending	0.91
Age – Spending	0.30
Experience – Income	0.88

If the redundancy threshold is:

[
T_c=0.85
]

Income and Spending would be considered highly correlated. The feature with the stronger relationship to the target should generally be retained.

Step 9: Removing Redundant Features

Suppose Income has a target correlation of 0.72 while Spending has a target correlation of 0.51. Since Income provides a stronger relationship with the target, Spending can be removed as a redundant feature.

This reduces the dimensionality while preserving most of the useful information.

Example

Suppose the original dataset contains:

Age

Income

Spending

Education

Experience

Customer_ID

Random_Code

After analysis:

Customer_ID → Identifier, remove

Random_Code → Irrelevant, remove

Spending → Highly correlated with Income, remove

Education → Low target relevance, remove

Age → Retain

Income → Retain

Experience → Retain

The final feature set becomes:

Age

Income

Experience

Step 10: Model Validation

Feature selection should not be considered successful merely because the number of attributes has decreased. The selected features must be evaluated using a machine learning model.

For example, first train a baseline model using all features.

Original Features

↓

Machine Learning Model

↓

Accuracy = 88%

Then train the same model using the selected features.

Selected Features

↓

Machine Learning Model

↓

Accuracy = 91%

Since the selected feature set produces higher accuracy while using fewer attributes, the feature selection process can be considered successful.

Performance Metrics

For classification problems, the selected feature set can be evaluated using:

$$
Accuracy=\frac{TP+TN}{TP+TN+FP+FN}
$$

$$
Precision=\frac{TP}{TP+FP}
$$

$$
Recall=\frac{TP}{TP+FN}
$$

$$
F1=\frac{2(Precision)(Recall)}
{Precision+Recall}
$$

For regression problems, metrics such as MAE, MSE, RMSE, and R^2 can be used.

Dimensionality Reduction

The effectiveness of feature selection can also be measured using the dimensionality reduction percentage.

$$
Reduction=
\frac{Original\ Features-Selected\ Features}
{Original\ Features}
\times100
$$

For example, if the original dataset contains 20 features and only 8 are retained:

$$
Reduction=
\frac{20-8}{20}\times100
$$

$$
=60\%
$$

Therefore, the dimensionality has been reduced by **60%**.

Validation of the Selected Features

The final feature set should be validated using cross-validation rather than relying on a single train-test split. Cross-validation evaluates whether the selected features consistently improve model performance across different subsets of the data. Feature selection itself should ideally be performed inside each training fold to avoid data leakage.

Avoiding Data Leakage

A major consideration during feature selection is data leakage. Correlations and thresholds should be calculated using only the training data when evaluating a model. The test dataset must remain unseen until final evaluation. If information from the test set is used during feature selection, the reported model performance may be artificially high and may not represent performance on unseen data.

Advantages of Feature Selection

Feature selection reduces the number of input variables, decreases computational cost, improves model interpretability, reduces noise, can reduce overfitting, and may improve prediction performance. A smaller feature set also makes data visualization and analysis easier.

Limitations

Correlation-based selection has limitations because correlation primarily measures linear relationships. A feature with low correlation may still be highly useful if its relationship with the target is nonlinear. Correlated features may also become useful when combined with other variables. Therefore, correlation analysis should be combined with model-based validation rather than being used as the only feature selection technique.

Conclusion

The proposed feature selection algorithm systematically reduces dataset dimensionality while attempting to preserve the most important predictive information. The process begins with data cleaning and normalization, followed by feature-target correlation analysis to identify irrelevant attributes. Feature-feature correlation is then used to detect redundant variables, and threshold-based filtering removes features that provide insufficient information. The resulting feature subset is validated using machine learning performance metrics and cross-validation to ensure that dimensionality reduction does not reduce predictive capability. Therefore, the proposed approach provides an efficient and practical method for reducing redundant and irrelevant features while improving model efficiency, interpretability, and potentially generalization performance.